Avraham “Abe” Bernstein | CV

Email: *Avraham DOT Bernstein PLUS cv AT gmail DOT com*
Tel/Whatsapp: *+972.54.641-0955*
City: *Jerusalem 9727433 ISRAEL*
Time Zone: *UTC +02:00/+03:00 (winter/summer)*
Shabbat Observant: *Not accessible electronically nor engaging in any business activities from Fri. evening (Jerusalem time) beginning 1 hour before sunset until Sat. night 1 hour after sunset, nor on Jewish holidays*
Linkedin: *https://www.linkedin.com/in/avrahambernstein/*
WWW Home Page: *https://www.avrahambernstein.com*
Most Recent Version of CV: *https://www.avrahambernstein.com/AvrahamAbeBernstein-CV.pdf*

Last Update: *2026-08-18*

1. Summary

I am a senior software architect with over 40+ years of **INNOVATION** in:

- compiler construction, and design of domain specific languages (DSL)
- design of command line interfaces (CLI) and application programming interfaces (API)
  - for example that are extremely useful for scripting and testing GUIs, and for communicating with AI agents
- automated source code refactoring of C (2018), C++ (2014), C# (ECMA334), and Java (SE8)
  - via the AST-XML conversion utilities srcML and Beautiful Soup
- unique source code obfuscation techniques that greatly reduce the reverse engineering "attack surface" of linux apps (both workstation and embedded)
- simplify the design of source code with the assistance of various macro preprocessors and template engines
- algorithm design
- multiple patents and inventions in many different application domains

I worked at an **expert level** in many industries including:

- cybersecurity
- automotive
- internet TV
- accessibility
- bioinformatics

I am an expert generalist and autodidact polymath who thrives on technically challenging projects. I worked with CTO groups to solve problems that initially had **AMBIGUOUS** problem specifications in order to build prototypes and minimum viable products (MVPs). I was able to refine the problems in order to build commercially useful and *testable* solutions. I prefer to work with and to mentor small teams.

On account of my age, I find that working as a consultant-contractor is the best fit for most employers. And for HR executives, here is a synopsis of a financial study from Stanford Univ. on the benefits of hiring older employees .

Core Skills & Tools

- **Languages:** C, Python, Tcl, Jinja2, Pyexpander, Bash & Posix CLI Commands, HTML, XML, Markdown, WASM.
- **Technologies:** srcML (acquired commercial license), Beautiful Soup, Linux, ELF, Misra C, GCC, Clang, Zydis, Pandoc, Obsidian, YAML & PKL.
- **Domains:** Compiler Design, Domain Specific Languages (DSL), Code Refactoring and Obfuscation (= anti-reverse engineering), Cybersecurity, Reverse Engineering, Embedded Systems, Accessibility, Automotive Software, Factory Automation, Bioinformatics, Network Protocols.

2. Inventions (Reverse Chronological Order)

1. Showed how srcML combined with Python Beautiful Soup could automatically refactor "C" source code for greatly improving the efficiency of automotive software updates, and for cybersecurity obfuscation.
2. Invented Linux cybersecurity techniques, especially to minimize the attack surface of `.so` (DSO) files that supply no formal exports, efficiently shroud system calls and strings and constants and randomize the stack, and thus are extremely difficult to reverse engineer. See short technical description.
3. Patented FLASH techniques for implementing S/W updates on small FLASH devices, and small RAM for the automobile industry which are plagued with minimal FLASH and RAM in order to reduce the materials cost of the millions of vehicles that they produce.
4. Invented a technique for *dynamically* loading a large static FLASH database into an embedded system that could be larger than RAM that does not violate any Misra C restrictions, with the help of preprocessing tools such as *Jinja2* or *Pyexpander*.
5. Invented a simple technique to obfuscate photographs using GIMP filters while remaining easily recognizable by a young child, but not recognizable by photographic database software.
6. Invented highly accurate fuzzy logic classifier used to discover rooted *Android* devices.
7. Showed how trivial Virtual Machine (VM) attacks could disrupt *Digital Rights Management (DRM)* protection, and how the QEMU VM could

disrupt the cryptographic nonce mechanism which allowed subscriber IDs to be shared by a confederacy of pirates.

8. Invented a generic font modification technique in order to make them understandable by dyslexics.
9. Invented a technique that enabled the blind to "see" and navigate digital maps and to explore mathematical functions on standard PCs and smartphones with a sound card and the addition of a consumer grade graphics tablet via the use of custom S/W that runs on a standard web browser with SVG support.
10. Invented *bioinformatic* PCR algorithms that (1) can accurately detect the "Ct" of an *assay* with high levels of *inhibition* via the use of an AI threshold algorithm instead of the classic functional analysis technique, and (2) can trivially clean the *systematic noise* from an *assay*, especially useful when the *assay* contains significant *inhibition*. Note that the project took only 2 months, even though I had zero formal background in microbiology and genetics. I received intensive mentoring from domain experts. I saved the client's project from bankruptcy.
11. Invented a network protocol for hybrid cable modems with a dialup upstream and RF downstream, where the "edge router" controlled access to both the dialup and RF networks, which enabled the edge router's ARP table to be dynamically modified when a modem logged on and off.
12. Designed a super efficient cable modem network laboratory which multiplied by a factor of 24 how many modems could be attached to a single PC via multiplexing with a dynamically programmable layer-2 switch.
13. Designed a GCC port for Forth-like CPUs that effectively allowed an unlimited number of registers.
14. Designed Domain Specific Languages (DSL) for Quality Assurance (QA) projects that allowed them to be tested with scripts. Examples included satellite communications, client movie players, and milspec testing laboratories. The scripts exploited flaws in the satellite software design. The satellite scripts were used to implement sanity tests on the developer's desktop prior to code check-in which significantly reduced QA testing. The mil-spec scripts replaced 3+ meter high stack of test docs, and allowed the system engineer to write simple *ad hoc* tests.
15. Invented a DSL for a VLSI toolchain that clock-accurately emulated the CPU in "C" including its instruction pipeline. The "C" emulator ran 100 times faster than the electrical (e.g. VERILOG) simulator. Therefore the DSL enabled instruction set sanity tests to be run immediately every time the architecture changed. The DSL solved one of the most painful problems in designing the assembler manually, namely detection of when the pipeline was broken which required programmer insertion of NOP instructions. For many months the VLSI architects changed the pipeline restrictions on a regular basis. Therefore DSL included a restriction language. It allowed the assembler to be built within 2 hours multiple times a week. The DSL was expanded to include a disassembler. Eventually the toolchain formed the backbone of a GUI debugger. The debugger incorporated programmable simulation of I/O ports. The debugger enabled the building of accurate applications many months before the physical CPU was produced (i.e. "taped out").
16. Invented a DSL which described an automated sintered metal blade production factory in complete detail. At that time commercial quality object oriented (OO) languages did not yet exist. But the factory contained about 50 object classes, e.g. workstations, stands, pallets, automatic guided vehicles, stacks, conveyors, cranes, etc. Interrupt routines were required for tool sharpening maintenance, oiling of parts that stayed too long in storage, etc. The DSL included a scheduling language. We designed an object oriented language (in Pascal) which completely described the factory. The factory was operational within a year. The factory engineer could make gross changes to the configuration file, while clerks had individual GUI screens to make spot changes. Originally I had zero knowledge of factory design, but after about 2 month of mentoring from an industrial engineer, we began to design the software.
17. Invented an application for quadriplegics that enabled them to access a PC with a CRT screen via a telescopically extended light pen (800 mm vs 5 mm distance) and via a sip-and-puff accessibility switch. The subject wears a head band to which the pen was attached. Designed a virtual screen interface which only popped up an individual virtual key when the pen hovered over it, leaving 95% of the screen still visible. The first user was a quadriplegic polio victim who was able to type 30 characters per minute, and became a financially independent book editor.
18. Invented many small CS algorithms over the years: hardened (minimized collisions) Adler-32 and new Adler-64 checksum; cryptographic quality key wrapper implemented in registers; *invertible* cross Hamming Weight transformation; cryptographic quality FFT *uniformly distributed* Hamming Weight-like primitive; binary expandable hash table (size `2^N`) that can grow without rehashing

3. Work Experience

2025: Independent: Founder & Principal Engineer Compiler & Obfuscation Tools Development. Download brochure.

2022-25: Aurora Labs, Tel Aviv: Senior Software Architect in CTO office for Digital Automotive Industry

2021: Morphisec, Beer Sheva: Senior Software Architect: Anti-Reverse Engineering Modifications to *Linux x64* Libc Kernel implemented with the help of the Zydis x64 disassembler

2021: Qedit, Tel Aviv: Consultant: WASM Cybersecurity for Financial Industry

2017-20: *Argus Cyber Security* (now PlaxidityX), Tel Aviv: Senior Researcher for Digital Automotive Industry

2013-17: Viaccess-Orca (subsidiary of Orange FR), Ra'anana: Cybersecurity Obfuscation Manager for Internet TV industry

2017 part-time: Independent: Dyslexic Accessibility

2015 part-time: Canary Mission, Jerusalem: Consultant: Internet Security "Hygiene"

2013 part-time: *NVT*, US (defunct): CTO: Agritech Startup for Cassava Production in Nigerian Jungle

2012: *Telequest*, Jerusalem: VP R&D: Automated Vehicle Navigation To Find Optimal Routes in City Traffic

2011: Syntezza Bioscience, Jerusalem: Consultant: Bioinformatic algorithms

2005-10: *NDS* (now Synamedia), Jerusalem: Senior Cybersecurity Researcher: Internet TV Industry

2002-03: *Virtouch*, Jerusalem (defunct): VP R&D: Blind Accessibility

1999-2004: *Vyyo*, Jerusalem (defunct): S/W Group Leader: RF wireless cable modem industry

1998: *Fourfold*, Jerusalem (defunct): Software architect: modified GCC compiler for massively parallel CPU for a Forth-like CPU with unlimited registers

1991-97: *Pitkha*, Jerusalem (defunct): CEO: Domain Specific Language (DSL) Architect

1990-91: Iscar-Matkash, Tefen: Software Architect: Factory Automation

1988-89: *Cubital* (subsidiary of Scitex), Herzliya:

- S/W R&D: Early 3D printing (*stereo-lithography*)
- Inventor: Quadriplegic Accessiblity for PCs

...1983-84: Elta IAI, Ashdod: junior embedded programmer: Lavi fighter plane

...1977: Ontario Energy Board (OEB) , Toronto

- While attending the MBA program at the Univ. of Toronto (see below), I was an intervenor in the *ECAP77* hearings on marginal cost pricing for electricity at Ontario Hydro . I was the first public interest intervenor in the history of the *OEB* to be awared costs. I published an oped about the hearings in Canada's then paper of record The Globe and Mail . With perfect hindsight the experience taught me a great deal, that I carried throughout my whole life, how a single individual could stand up and make a meaningful difference against government injustice.

4. Education

1979: *York University*, Toronto Canada: **MA Economics**, minor in Applied Mathematics

1973-78: *University of Toronto*, Canada: **BA** Undergraduate School of Arts & Sciences

- I did graduate studies at the Rotman Graduate School of Management (MBA program) and Graduate School of Engineering where credits were applied to my *MA Economics* above. After the *Ontario Energy Board* (OEB) hearings immediately above in 1977, I switched focus to engineering. I extraordinarily passed my economic compulsory examinations before I even started the *York Univ.* program, so the school allowed to me to take graduate level courses anywhere I chose.

5. Teaching & Mentorship

Part-time college and highschool instructor and mentor for computer science and mathematics. I am a strong advocate of pairing with domain experts to accelerate onboarding and innovation.

For younger children I tutor mathematics and music (via the free MuseScore application). I taught a 2nd grader the basics of exponents and logarithms within 30 minutes using a mathematical calculator application found on most smart phones. I kinesiologically taught my mildly ADHD 5th grade granddaughter a half year of fractions in two morning sessions at a bakery (beginning with a non-sweetened breakfast) by using a roll of 50 coins. Similarly I kinesiologically taught my 3rd grade grandson the basics of multiplication using floor tiles and masking tape. I remotely (via the free AnyDesk application) teach my 7th grade grandson computer and mathematical programming using Python turtle graphics. I am preparing my 11th grade granddaughter for her Israeli high-school matriculation examination in mathematics (4 units). (I expect/hope that she will get a grade of 90). I have a technique for teaching deep meditation within a few hours by using a free metronome application. I teach the children co-ordination and concentration exercises by using "boxer" jump rope techniques.

6. AI Experience

LLM

My last project at *Aurora Labs* (above) in 2025 required that I use an *LLM* to estimate the execution time required to run an assembly program on a (then) new automotive *Infineon* CPU by training with *JTAG* output. However I found that by creating a QEMU-like description of the instruction set in custom Python code **which included timing information**, I got much more accurate timing results. Unfortunately management was displeased with my approach because an *LLM* was new and sexy compared to *QEMU*. In one of my first major projects for *DSPG* (above) in 1992, I already had experience creating a clock accurate CPU emulator.

In general I find that running a pre-processing parser that *understands the structure of the raw text* used to "prompt" an LLM provides much better results than simply feeding an LLM raw text. For example a common problem is using an LLM to process a PDF file. The processing can be greatly improved when fed the Markdown text that was used to generate the PDF. When the original Markdown is not available then a FOSS program such as the *markdown-it* PDF parser which outputs Markdown enables the LLM to generate much more accurate and much less expensive results because the raw PDF file contains a huge amount of formatting "noise" which intentionally is not contained in the Markdown file. Similarly an LLM can understand computer source code far better when provided with its *AST* description (e.g. output by *SrcML* or *Tree-Sitter*) as opposed to the raw source code. And very recently (June 2026) Google has invented a v0.1 public domain Open Knowledge Format (OKF) language for describing an LLM Wiki comprised of human readable (industry standard) Markdown and YAML files that makes it more accurate and faster and more economical for the various LLM engines to contextualize knowledge. I am starting an *OKF* experiment using the *Obsidian* (2nd brain) wiki package combined with an *Ollama* local LLM.

Threshold Classifier

In 2011 I worked on a bioinformatics PCR project for *Syntezza Bioscience* (above). Their preliminary MRSA detector kit chemistry did not correctly remove impurities from patient samples. Therefore instead of their results showing a classic *sigmoidal* curve of a *bioassay*, their results looked like a random cloud, so the industry standard _PCR+ mathematical analysis of curve fitting and functional analysis would not work. Because their results were so utterly incorrect, they were on track for bankruptcy, from which my AI "threshold classifier" algorithm saved them. After discussing the matter with the company's chief scientist who was a microbiologist, he agreed with my observation that inhibition resulting from the impurities only reduces some, *but usually not all*, of the bioassay data points; regardless inhibition never increases their value. On each of the 3 PCR channels, I used only the single highest raw data point. (In hindsight, perhaps I should have used the average value less the standard deviation). I split the 800 samples in half, one half to be used for training and the other half to be used to test my predictions. All samples were initially tested by the gold standard analogue Petri dish analysis to determine whether or not they were MRSA positive. From the training samples I determined what was the lowest raw value of positive samples, and the highest raw value of negative samples (for each of the

3 PCR channels). Each test sample that exceeded the threshold value of the training data's MRSA positive minimal value was declared PCR positive, also as long as that sample did not fall below the threshold value of the training data's MRSA negative maximal value (for each of the 3 PCR channels). My predictions were 95% accurate. At a PCR MRSA "shoot out" at the prestigious St. Georges teaching hospital in London, my algorithm was a more accurate predictor than those of the big international pharmaceutical companies relying upon classical PCR mathematics. Note that the whole project took me just 3 months. Before that I had no university level background in biology, genetics, or bioinformatics.

7. To Learn

1. *FOSS* AI Ollama: a local AI LLM model that supports 32-64 GB RAM using an *x64* CPU in order to eliminate the exhorbitant cost of server based AI, and in order to protect my privacy. I am using local AI as an intelligent **assistant** where I note that AI is making intelligent comments about my code or documents, and **not** to automatically generate slop. And I am studying Google's Open Knowledge Format in order to improve how to more efficiently interact with AI software, also in conjunction with Obsidian OKF.
2. *FOSS* Neovim: a programming editor upgrade to Vim.
3. *FOSS* Tree-Sitter Abstract Syntax Tree (AST): a fast multi-language dynamic AST parser plugin for programming editors such as *neovim* (above). Also Emacs has excellent tree-sitter support.
4. *FOSS* Clang AST Library: a very large and more complex library for implementing `C/C++` AST static source code analysis and refactoring.
5. *FOSS* Zig: a new `C`-like programming language that can cross compile to `C` and that conforms to the industry standard `C` Application Binary Interface (ABI). Has many new language features such as much better dynamic memory allocation mechanism, generic types, and especially compile time functions instead of preprocessing macros.
6. *FOSS* Mimalloc: a better dynamic memory allocator that is a drop-in replacement for the `C` malloc that can be dynamically implemented system-wide via `LD_PRELOAD`.
7. *FOSS* Musl: a static replacement for *Linux* libc with better security because no need for loading dynamic shared libraries (i.e. `.so` files).
8. NeuroSky and Muse 2: small inexpensive (~ USD$250) **non-invasive** brain computer interface (BCI) devices that enable brain waves to control a virtual keyboard and mouse. Physically these devices are similar to a plastic woman's hair band worn across the forehead. Especially important is that these devices do not require pasting any electrodes on the scalp. The devices can be configured to allow a disabled user (i.e. who does not have the physical ability to control a physical keyboard but otherwise have full cognition) to control a "virtual" keyboard. They can be used for rehabilation projects. Unfortunately here in Israel there are hundreds of soldiers who have been disabled by the wars over the past few years, and they provide an excellent laboratory environment to create such software tools.